

Hochschule Bielefeld
Fachbereich Ingenieurwissenschaften und Mathematik
Betriebssysteme

Analyse von Speicherzuteilungsverfahren

Philipp Bleimund (50033807)

7. Juli 2026

Inhaltsverzeichnis

1	Implementation der Algorithmen	1
1.1	first fit	1
1.2	next fit	1
1.3	best fit	2
1.4	worst fit	3
2	Vergleich	3

1 Implementation der Algorithmen

Im Folgenden sind die verwendeten Speicherverhaltensalgorithmen dargestellt.

1.1 first fit

Listing 1: first fit C-Implementation

```
1 unsigned firstFitAlloc(unsigned blockSize) {
2     unsigned address = 0;
3
4     // loop over free space to check if there is a block bigger than blockSize
5     for (unsigned i = 0; i < freeList.size(); i++) {
6         if (freeList[i].size >= blockSize) {
7             // free space found
8             address = allocBlock(i, blockSize);
9             break;
10        }
11    }
12
13    return address;
14 }
```

Bei first-fit wird der erste verfügbare Speicherblock verwendet. Dazu wird in der Implementation 1 über den Vektor *freeList* iteriert. Wird ein Block gefunden, welcher gleich groß oder größer ist, wird dieser alloziert und die Suche abgebrochen.

1.2 next fit

Listing 2: next fit C-Implementation

```
1 unsigned nextFitAlloc(unsigned blockSize) {
2     unsigned lastAddressNextFit = 0;
3     unsigned address = 0;
4
5     if (freeList.empty())
6         return address;
7
8     if (lastAddressNextFit == 0) {
9         lastAddressNextFit = freeList.front().address;
10    }
11
12    // search for lastAddressNextFit
13    unsigned lastAddress_idx = 0;
14    for (unsigned i = 0; i < freeList.size()-1; i++) {
15        if (freeList[i].address >= lastAddressNextFit && lastAddressNextFit <=
16            freeList[i+1].address) {
17            lastAddress_idx = i;
18            break;
19        }
20    }
21
22    // loop over free space to check if there is a block bigger than blockSize
23    // start at last Address
```

```

23     unsigned n = freeList.size();
24     for (unsigned j = 0; j < n; j++) {
25         unsigned i = (lastAddress_idx + j) % n;
26         if (freeList[i].size >= blockSize) {
27             lastAddressNextFit = freeList[i].address;
28             address = allocBlock(i, blockSize);
29             break;
30         }
31     }
32     if (address == 0)
33         lastAddressNextFit = 0;
34
35     return address;
36 }

```

Next-fit funktioniert wie first-fit. Der Unterschied ist, dass next-fit die letzte allozierte Adresse speichert und diese als Startpunkt der first-fit Suche verwendet. In der Implementation von next-fit 2 wird dies über die globale Variable `lastAddressNextFit` realisiert. Es wurde sich für die Speicherung der Adresse und nicht des letzten Indexes entschieden, da der Index innerhalb des Vektors durch die zufällige Freigabe von Speicherblöcken verändert werden kann.

Daher wird in den Zeilen 11 bis 17 die Adresse in einen Index umgewandelt. Da sich durch die vorherige Allokierung die Adressen in *freeList* ändern, wird nach einer passenden Lücke für die alte Adresse gesucht. Anschließend wird der nächste freie Block in dem Vektor *freeList* gesucht und alloziert. Über die Rechenvorschrift $(lastAddress_idx + j) \% n$ wird dafür gesorgt, dass die Suche bei *lastAddress_idx* startet und wieder endet.

1.3 best fit

Listing 3: best fit C-Implementation

```

1  unsigned bestFitAlloc(unsigned blockSize) {
2      unsigned address = 0;
3
4      // loop over free space to find smallest fitting block
5      int best = -1;
6      unsigned lastFit = UINT_MAX;
7      for (unsigned i = 0; i < freeList.size(); i++) {
8          int nextFit = freeList[i].size - blockSize;
9          if (nextFit >= 0 && (unsigned)nextFit < lastFit) {
10             best = i;
11             lastFit = nextFit;
12         }
13     }
14     if (best == -1)
15         return 0;
16     address = allocBlock(best, blockSize);
17
18     return address;
19 }

```

Best-Fit sucht einen freien Speicherblock, welcher nach dem Allokieren den kleinsten neuen Speicherblock erzeugt. Die Implementation 3 erreicht dies durch eine Suche über *freeList*. Bei jedem Element wird verglichen, ob der Speicherblock groß genug ist und ob der überbleibende Block kleiner ist als der letzte beste Wert. Anschließend wird der gefundene Block alloziert.

1.4 worst fit

Listing 4: worst fit C-Implementation

```
1 unsigned worstFitAlloc(unsigned blockSize) {
2     unsigned address = 0;
3
4     // loop over free space to find biggest fitting block
5     int best = -1;
6     unsigned lastFit = 0;
7     for (unsigned i = 0; i < freeList.size(); i++) {
8         int nextFit = freeList[i].size - blockSize;
9         if (nextFit >= 0 && (unsigned)nextFit > lastFit) {
10             best = i;
11             lastFit = nextFit;
12         }
13     }
14     if (best == -1)
15         return 0;
16     address = allocBlock(best, blockSize);
17
18     return address;
19 }
```

Worst-Fit sucht einen freien Speicherblock, welcher nach dem Allokieren den größten neuen Speicherblock erzeugt. Die Implementation 4 ist gleich der Implementation von best-fit 3 mit dem Unterschied, dass die Suchbedingung größere überbleibende Blöcke bevorzugt.

2 Vergleich

In Abb. 1 und Abb. 2 sind die Ergebnisse vom Test der Algorithmen dargestellt.

Im Vergleich der Algorithmen ist zu erkennen, dass first-fit, next-fit und best-fit ähnliche Ergebnisse liefern. Wohingegen next-fit etwas schlechter als first-fit und best-fit ist. Worst-fit ist der einzige Algorithmus, welcher schlechter abschneidet als die anderen.

In Abb. 3 wurde zusätzlich zur maximalen Größe einer einzelnen Allokierung auch der verfügbare Speicher variiert. Es ist zu erkennen, dass best-fit in den meisten Fällen der beste Algorithmus ist. Best-fit ist in 11113 Kombinationen der beste Algorithmus. Am schlechtesten schneidet next-fit mit einem Wert von 684 ab. First-fit und worst-fit schneiden ähnlich mit 1659 und 993 Kombinationen etwas besser als next-fit ab. Die weiße Fläche stellt alle Kombinationen dar, in denen es mehr als zwei beste Algorithmen gibt und dominiert mit 25152 Kombinationen. Die Transparenz gibt an, wie knapp ein Algorithmus gewonnen hat. Dadurch ist zu erkennen, dass bei einem Verhältnis von kleiner Allokierungsgröße und großem vorhandenen Speicher best-fit mit einem größeren Abstand der beste Algorithmus ist.

Es ist zu erwarten, dass first-fit und next-fit ähnliche Werte erreichen, da sie ähnliche Implementationen haben. Worst-fit schneidet am schlechtesten ab.

Abb. 4 vergleicht die benötigte Zeit von jedem Algorithmus zu den erreichten maximalen Allokierungen. Dabei ist zu erkennen, dass worst-fits Ziel vom Reduzieren der Zeit im Vergleich zu best-fit klappt. First-fit hingegen erreicht fast so viele Allokierungen wie best-fit, in einer geringeren Zeit.

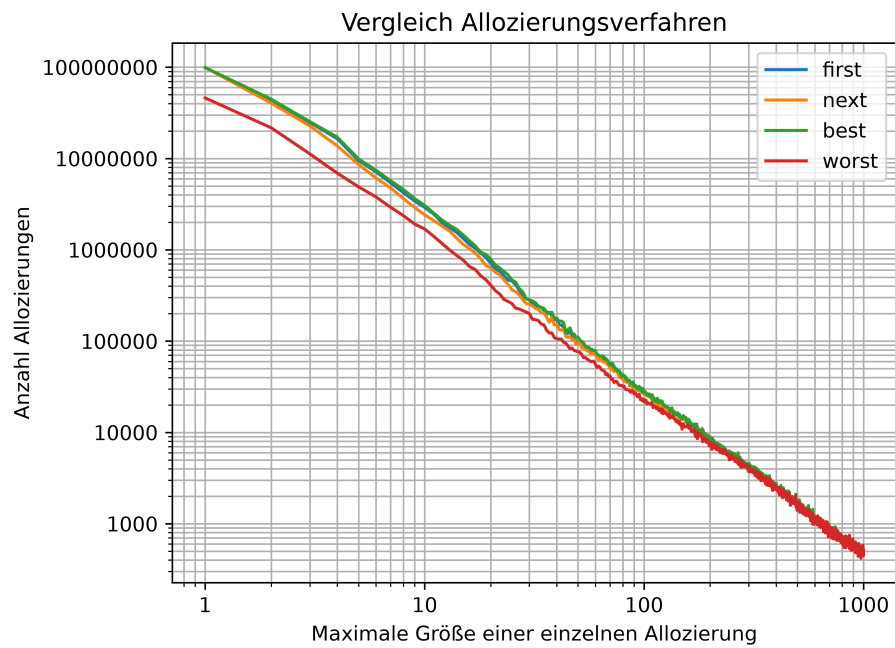


Abbildung 1: Vergleich Algorithmen in loglog Darstellung.

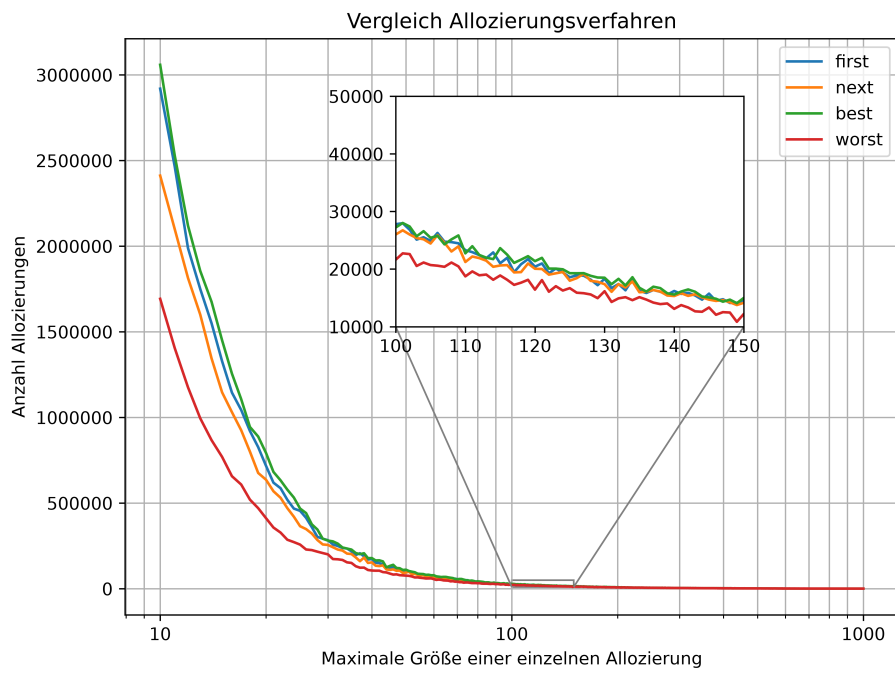


Abbildung 2: Vergleich Algorithmen in einfacher log Darstellung.

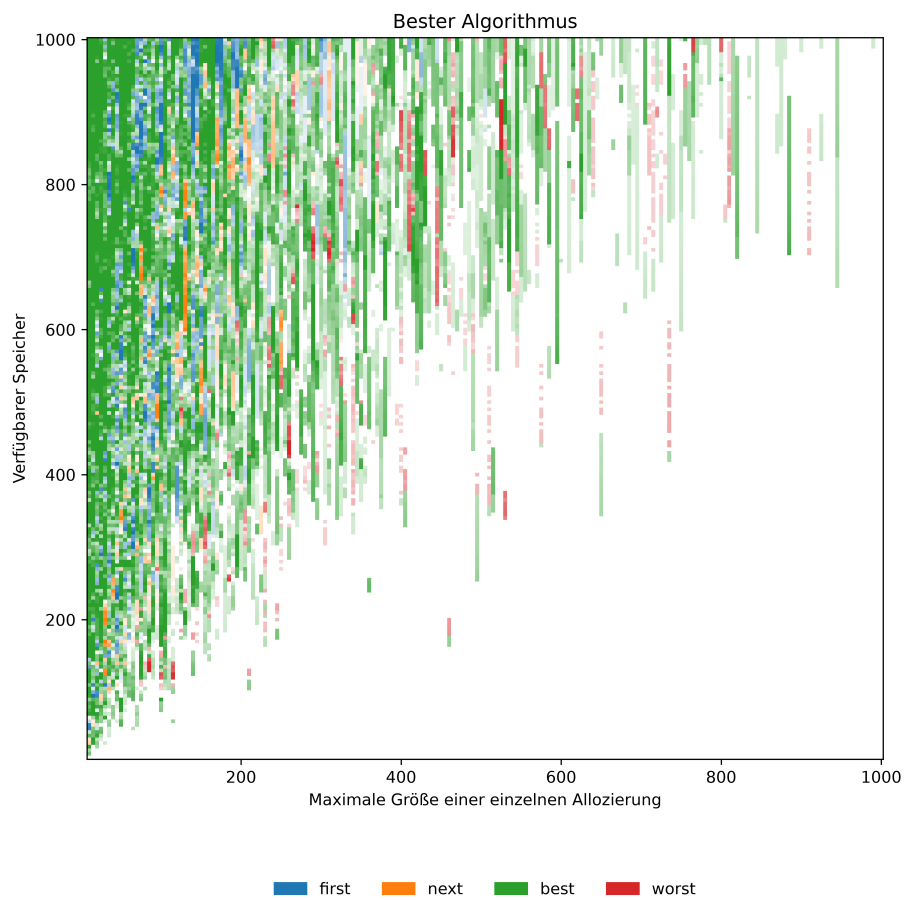


Abbildung 3: Vergleich Algorithmen als Heatmap.

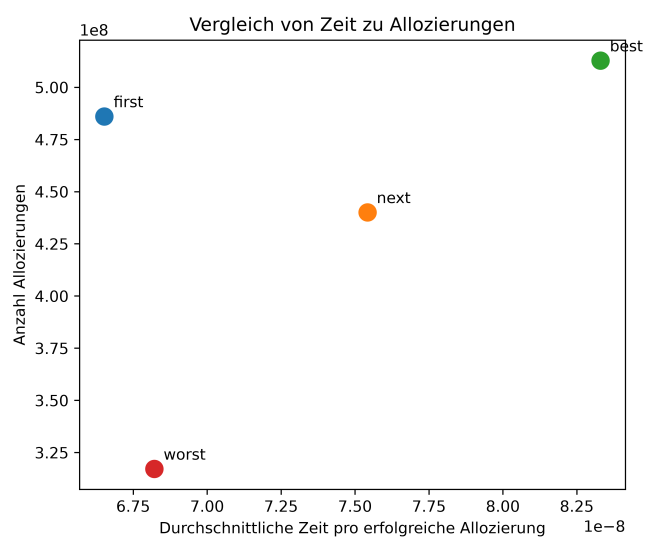


Abbildung 4: Vergleich Algorithmen und deren benötigte Zeit.